Helm: chart, template, values và release

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Helm chart và cấu hình theo môi trường

Manifest Kubernetes thuần mô tả từng resource. Helm chart là gói template và metadata tạo ra manifest từ **values**. Một chart tối thiểu có **Chart.yaml**, **values.yaml** và **templates/**. **helm template** giúp xem YAML sẽ gửi tới cluster; **helm lint** kiểm tra một số lỗi chart; **helm upgrade --install** áp dụng release. Cần phân biệt version của **chart** với version của **ứng dụng/image**. Tăng chart version khi template hoặc giá trị mặc định thay đổi; giữ image digest riêng để biết chính xác bản chạy.

Values theo môi trường nên chứa những khác biệt thật như replica count, hostname, resource requests, endpoint. Đừng copy một chart thành ba thư mục staging/preprod/prod rồi sửa tay; chúng sẽ lệch dần. Không đưa password vào **values.yaml** trong Git. Với secret, chọn cơ chế phù hợp như external secret/Vault/Key Vault và kiểm tra quyền service account. **helm template** có thể hiển thị secret nếu đưa secret vào values; vì vậy log build cũng cần tránh in giá trị nhạy cảm.

Lab 2 - Chart tối thiểu cho Notes API

Tạo thư mục **chart/notes-api**; định nghĩa Deployment và Service từ **lesson/labs/kubernetes-local/web.yaml** làm mẫu. Cho **image.repository**, **image.digest**, **replicaCount**, port và resources vào values. Chạy **helm lint chart/notes-api** và **helm template notes chart/notes-api**; đọc đầu ra, xác nhận Service selector khớp label pod, containerPort đúng, và image trỏ đúng digest. Đổi **replicaCount** từ 1 thành 2 rồi render lại, giải thích **chỉ desired replicas đổi**; chưa có bằng chứng cluster đã scale. Nếu chưa cài Helm, phác cấu trúc và đọc output ví dụ từ tài liệu chính thức trước. Dọn dẹp release lab bằng **helm uninstall notes** nếu đã cài trên cluster local.

Lỗi có chủ đích: đặt selector **app: notes-v2** nhưng pod label **app: notes**. Pod vẫn Running, Service không có endpoint. Dùng **kubectl get endpoints**, **kubectl describe service** và **kubectl get pods --show-labels** để chứng minh, sau đó sửa template.

Chart gồm những gì và được render ra sao?

Một chart là thư mục có **Chart.yaml** (tên và version), **values.yaml** (giá trị mặc định), **templates/** (manifest có biểu thức) và có thể có **charts/** dependency. Helm kết hợp chart + values để render YAML rồi gửi tới Kubernetes API. Cần đọc **YAML sau render**, không chỉ đọc template: một dấu cách, **default** hoặc điều kiện **if** có thể tạo resource khác bạn nghĩ. **helm lint** phát hiện một số lỗi cấu trúc; **helm template** cho đầu ra; Kubernetes API và admission policy còn có thể từ chối, và ứng dụng vẫn có thể lỗi sau khi resource được chấp nhận.

Phân biệt ba version: chart **version**, **appVersion** mô tả ứng dụng, và image tag/digest thực triển khai. **appVersion** không tự khóa image. Với release có thể kiểm chứng, pipeline ghi commit + chart version + image digest + values/environment. Hạn chế dùng **latest**; một tag có thể bị ghi đè, làm rollback không tái tạo được.

Lab thực hành cụ thể với Notes API

Tạo chart **lesson/labs/helm-notes/** gồm **Chart.yaml**, **values.yaml**, **templates/deployment.yaml**, **templates/service.yaml**. Đặt **replicaCount: 1**, image lab và port 8000. Chạy **helm lint lesson/labs/helm-notes**; tiếp theo **helm template notes lesson/labs/helm-notes**. Trong output, kiểm tra: selector Deployment khớp label Pod; selector Service cũng khớp; containerPort và targetPort cùng 8000; resource requests/limits có ý nghĩa; image pull policy phù hợp cluster local. Chart dùng **emptyDir** cho SQLite **chỉ để học**: xóa Pod sẽ mất dữ liệu; production cần thiết kế persistence khác. Nếu chưa có image notes-api trong cluster, chỉ render/lint; đó là bài kiểm tra template, không giả vờ đã triển khai ứng dụng.

Thay **replicaCount=2** bằng **--set replicaCount=2**, render lại và so output. Thay **image.tag** bằng một tag không tồn tại rồi triển khai trên cluster lab (nếu có), quan sát Pod event. **helm status** có thể báo release đã tạo trong khi Pod chưa Ready; **kubectl rollout status**, Service endpoints và smoke test trả lời lớp vận hành. Dọn dẹp bằng **helm uninstall notes -n <namespace>** sau khi xác nhận đúng context.

Các file tối thiểu

```
helm-notes/
  Chart.yaml
  values.yaml
  templates/deployment.yaml
  templates/service.yaml
```

Trong template, dùng một nhãn ổn định như **app.kubernetes.io/name: notes-api** cho cả Deployment và Service. Values môi trường chỉ ghi sự khác biệt: hostname, replica count, resource, image digest, endpoint. Không copy cả chart cho staging và production; bản sao sẽ lệch. Không commit secret vào values; **helm template** và **helm get values** có thể làm lộ dữ liệu tùy cách dùng.

Upgrade, lịch sử và rollback

helm upgrade --install tạo hoặc cập nhật release. Helm có release revision/history và có thể **helm rollback** về revision trước; cần hiểu rollback này áp lại manifest/values, không tự rollback dữ liệu đã đổi. Nếu chart thay PVC hoặc thực hiện hook migration DB, phải có kế hoạch cụ thể. **--atomic** hỗ trợ rollback trong một số tình huống upgrade lỗi nhưng không thay smoke test nghiệp vụ hay xử lý schema. Đọc diff/manifest trước, nhất là thay đổi xóa resource, Service, PVC và quyền RBAC.

Case: phiên bản mới có readiness 200 nhưng POST ghi DB 500. Helm release thành công; rollback image có thể cứu nếu schema tương thích. Nếu migration đã xóa cột cũ, rollback app sẽ vẫn lỗi. Cần release schema theo expand/contract và backup/restore được diễn tập trước khi đổi dữ liệu rủi ro. Câu trả lời phỏng vấn nên tách Helm release state với trạng thái sức khỏe ứng dụng.

Chart dependency và trách nhiệm vận hành

Chart dependency giúp cài kèm service, nhưng đừng mặc định gom database production vào chart của app. Vòng đời database, backup, upgrade và permission thường khác app. Trong microservices, chart chung có thể giảm lặp nhưng nếu template quá nhiều điều kiện, việc review trở nên khó. Dùng cấu trúc nhỏ, tên rõ, values có schema/kiểm tra, và test render cho mỗi môi trường. Nếu dùng Argo CD để deploy chart, xác định ai là owner đồng bộ; tránh vừa **helm upgrade** thủ công vừa để Argo tự kéo về khác.

Tư duy đúng: chart là chương trình tạo manifest

Helm chart không phải "một file YAML lớn". Chart chứa **Chart.yaml** metadata, **values.yaml** mặc định, thư mục **templates/** có Go templates và có thể có dependencies. Khi render, Helm kết hợp chart, values của môi trường, CLI overrides và khả năng cluster để tạo manifest Kubernetes. Kubernetes chỉ thấy **manifest cuối cùng**; nếu Pod lỗi, phải truy từ values qua template đến YAML đã áp. Một values key có tên đẹp không bảo đảm template thực sự dùng nó. Đọc rendered output là bước quan trọng hơn học thuộc **helm install**.

Ví dụ **replicaCount: 2** trong values và template dùng **{{ .Values.replicaCount }}** cho Deployment. Nếu file production đặt **replicas: 3** thay vì **replicaCount: 3**, Helm có thể vẫn render 2 mà không báo lỗi nếu chart không có schema/validation. Vì vậy review values cần kèm **helm template** hoặc **helm lint** và kiểm tra output theo điều kiện dự kiến. **values.schema.json** có thể bắt một số lỗi kiểu/dải giá trị, nhưng không thay test logic và Kubernetes validation.

Thứ tự ghi đè và ba số phiên bản

Chart có **chart version** (phiên bản gói), **appVersion** (metadata mô tả ứng dụng) và image tag/digest thực trong template. Chúng không tự đồng bộ. Một chart version đổi vì sửa probe mà app image không đổi; một image đổi mà chart version có thể chưa đổi trong chart nội bộ. Với release có thể tái hiện, ghi chart source/version, values cụ thể, image digest và revision Git. Nhiều file **-f** được kết hợp theo thứ tự, file sau có thể ghi đè file trước; CLI **--set** cũng có thể làm cấu hình khác Git. Trong production nên giảm tham số thao tác tay khó audit.

Lab đọc chart Notes API một cách có bằng chứng

Điều kiện: Helm CLI và kubectl đã cài; dùng chart trong **lesson/labs/helm-notes**, không dùng cluster production. Chạy **helm version** và ghi major version vì Helm 4 đã xuất hiện trong tài liệu hiện hành; tra cú pháp đúng phiên bản CLI của bạn. Trước khi render, dự đoán số Deployment, Service, replica, image, port và probe mà chart tạo. Chạy **helm lint lesson/labs/helm-notes** và **helm template notes-lab lesson/labs/helm-notes -n devops-lab** rồi lưu output để đọc. **Kết quả dự kiến:** manifest Kubernetes có các giá trị từ chart; lint có thể không chứng minh image tồn tại hoặc Pod Ready.

Tạo một file values lab riêng đổi replica từ 1 sang 2 và image/tag hợp lệ theo chart. Render lại với **-f <file-values-lab>**; so diff đầu ra, chỉ rõ trường nào đổi. Nếu chart dùng một key khác dự đoán, quay lại template/values và giải thích. Thử cố ý viết sai tên key trong **file lab**, render lại: nếu output không đổi, đây là ví dụ silent misconfiguration. Đừng áp lên cluster trước khi nhìn ra sự khác biệt. Nếu chart có

schema chặn lỗi, ghi thông báo và giải thích lớp bảo vệ đó.

Khi đã có cluster kind riêng và context đúng, **helm install notes-lab lesson/labs/helm-notes -n devops-lab --create-namespace** theo cú pháp phiên bản của bạn. Xem **helm status**, **helm get manifest**, **kubectl get deploy,svc,pods -n devops-lab**, **kubectl rollout status**. Sau một thay đổi values, dùng **helm upgrade**, đọc **helm history** và quan sát resource diff. Dọn bằng **helm uninstall notes-lab -n devops-lab** rồi xác nhận resource còn hay không; namespace/PVC/CRD có thể có vòng đời riêng. Không giả định uninstall là xóa dữ liệu hoàn toàn.

Template logic: dễ tiết kiệm sao chép, cũng dễ giấu lỗi

Go template có **if**, **range**, **include**, helpers và pipeline. Helper cho labels/tên thống nhất là tốt, nhưng lồng nhiều điều kiện khó kiểm tra mọi tổ hợp. Một chart có 12 boolean có thể tạo nhiều biến thể; không phải biến thể nào cũng được CI render. Định nghĩa cấu hình được hỗ trợ, test ít nhất values mặc định và từng môi trường thật. Nếu một tùy chọn tắt Service nhưng Ingress vẫn trỏ vào Service đó, render có thể hợp lệ về cú pháp nhưng app không phục vụ. Một chart tốt giảm bất ngờ, không tối đa hóa số lựa chọn.

Không đưa mật khẩu rõ vào **values.yaml** commit Git. Helm release metadata có thể lưu rendered manifest và values theo backend/cơ chế phiên bản, nên quyền đọc release cũng cần xét. Dùng secret manager/External Secrets hoặc tham chiếu Secret được cấp theo quy trình; nếu chart có khả năng tạo Secret, hiểu rõ dữ liệu đi qua CI/log/history. **helm template** output có thể chứa secret, vì vậy không đính kèm nguyên bản vào PR/log public.

Upgrade, rollback và quan hệ với Argo CD

Helm upgrade thay resource theo manifest mới; Kubernetes tiếp tục rollout. **helm upgrade** thành công ở cấp API không chứng minh readiness hoặc chức năng. **helm rollback** khôi phục release revision Helm theo khả năng công cụ nhưng không rollback DB migration, dữ liệu hay hệ thống ngoài chart. Nếu Argo CD quản lý một Application từ Helm chart, Argo render chart rồi áp manifest; không nên tự chạy **helm upgrade** vào cùng resource và kỳ vọng Argo giữ sửa tay. Quy định rõ ai là owner của release: CLI Helm hay GitOps controller.

Ca lỗi có lời giải: sau upgrade, Service vẫn tồn tại nhưng không có backend. Đọc rendered Deployment/Service: selector có còn khớp Pod labels không? Pod có Ready không? Probe path/port từ values có đúng không? Nếu chart template đổi label nhưng Service selector dùng helper cũ, Kubernetes chấp nhận hai resource mà traffic vẫn lỗi. Sửa chart, render, so diff, deploy staging rồi smoke test; tăng replica không sửa selector.

Câu hỏi: vì sao chart version 2.0 không chắc app image là 2.0? Vì version gói và image reference là trường khác. **Câu hỏi:** vì sao **helm lint** xanh chưa đủ? Nó kiểm tra phần cấu trúc/template nhất định, không chứng minh image pull, RBAC, storage hay nghiệp vụ. **Câu hỏi:** rollback release có xóa bảng DB mới? Không; DB nằm ngoài phạm vi bản manifest của chart và cần migration/restore plan riêng.

Bạn đạt bài khi có thể đọc chart từ values tới manifest, giải thích một diff upgrade, làm lỗi key/selector trong lab và khôi phục. Đối chiếu Helm chart guide và Helm chart best practices theo phiên bản CLI đang dùng.

Đọc template bằng một ví dụ nhỏ

Giả sử **values.yaml** có **service.port: 8080**, **service.targetPort: 8000** và Deployment container nghe 8000. Service template lấy **{{ .Values.service.port }}** và **{{ .Values.service.targetPort }}**; sau render, client trong cluster gọi Service:8080, proxy tới Pod:8000. Nếu người sửa values đặt **targetPort: 8080** nhưng process vẫn nghe 8000, manifest hoàn toàn hợp lệ và Pod Ready có thể vẫn đúng nếu probe gọi cổng khác; request qua Service thất bại. Vì thế khi review chart, đọc một **đường request trọn vẹn**: Ingress backend port -> Service port -> targetPort -> process listen, cộng selectors/labels. Một chart chỉ "lint xanh" không chứng minh đường này.

Trước khi deploy, in manifest cho staging và production rồi so: image digest, replicas, resources, probes, env, Secret refs, Service/Ingress, RBAC, PVC. Nếu chỉ khác **replicaCount**, diff cần nhỏ và giải thích được. Nếu production values vô tình đổi **service.type** thành LoadBalancer, diff có thể tạo exposure/cost mới. Review **rendered manifest** làm lộ tác động thật của values, không chỉ file values vài dòng. Đối với chart bên thứ ba, đọc changelog và diff templates khi nâng version; default values mới có thể đổi hành vi dù file override của bạn không đổi.

Chart dependency và supply chain

Một chart có thể phụ thuộc subchart DB/Redis. Với production, hỏi có nên để chart ứng dụng tự cài DB stateful hay dùng DB được đội dữ liệu quản lý. Subchart tiện lab nhưng có thể kéo storage/backup/upgrade vào vòng đời app release ngoài ý muốn. **Chart.lock** và version constraint giúp tái hiện dependency theo cơ chế Helm; vẫn cần kiểm soát repository nguồn và provenance/integrity nếu tổ chức yêu cầu. Một chart public có thể đổi template hoặc image reference giữa hai lần kéo; pin version và review artifact.

Bài tập: chart Notes API bật DB subchart ở staging, production dùng DB managed. Viết hai values files, render và kiểm tra production không tạo StatefulSet/PVC ngoài ý muốn, app env trỏ DB production qua Secret reference, network policy chỉ cho đường cần thiết. Nếu chart không hỗ trợ rõ hai mode, quyết định sửa chart/đơn giản hóa thay vì chồng hàng loạt **--set** khó hiểu.

Release history có giới hạn

Helm lưu lịch sử release để xem revision và rollback theo cơ chế phiên bản đang dùng. Lịch sử này không phải backup mọi thứ: PVC data, external DB, secret manager, image registry nằm ngoài chart. Nếu registry xóa image cũ, rollback manifest có thể kẹt **ImagePullBackOff**. Nếu DB migration đã xóa cột, app cũ rollback có thể crash. Kế hoạch release phải giữ artifact cần thiết, migration tương thích và kiểm tra sau rollback. Đặt retention lịch sử không quá thấp để còn truy vết, nhưng cũng xét storage/quyền đọc manifest có thể chứa dữ liệu nhạy cảm.

Ca có lời giải: helm upgrade thành công, Argo báo OutOfSync

Nếu Argo quản lý cùng app từ Git, một upgrade thủ công đổi live state ngoài Git. Argo so và báo drift; self-heal có thể đưa về Git. Không "sửa" bằng cách tắt Argo lâu dài. Xác định owner, đưa thay đổi chart/values vào Git, review, để Argo sync; nếu hotfix khẩn cấp, ghi incident và hợp thức hóa Git sau. Nếu release thật do Helm CLI quản lý chứ không qua Argo, kiểm tra vì sao Application Argo lại theo dõi cùng resource và loại bỏ ownership trùng.

Ca có lời giải: một giá trị boolean bị parse sai

Bạn muốn **feature.enabled=false** nhưng dùng **--set feature.enabled="false"** hoặc file values với kiểu string ở nơi chart kỳ vọng boolean. Tùy cách parse và template, điều kiện có thể vẫn bật hoặc render sai. Dùng file YAML có kiểu rõ, schema chart nếu có, **helm template** và kiểm tra resource sinh ra. Trong phỏng vấn, nói "kiểu dữ liệu values" và output manifest, không chỉ "Helm bug".

Bạn đạt phần này khi có thể cầm một chart chưa thấy, render hai môi trường, giải thích một diff có tác động bảo mật, và nói vì sao rollback chart chưa đủ cho dữ liệu. Đọc Helm template guide theo phiên bản của bạn.

Thử thách cuối: tạo một thay đổi values chỉ định **resources.requests.memory** thấp hơn mức app thực dùng. Trước khi apply, dự đoán scheduler sẽ dễ đặt Pod hơn nhưng runtime có thể OOM nếu limit cũng thấp hoặc node quá tải; render manifest để biết requests và limits thực. Nếu rollout kẹt, đọc Pod Events/status chứ không quy mọi lỗi cho Helm. Helm tạo YAML; Kubernetes quyết định schedule/run; ứng dụng quyết định phục vụ nghiệp vụ. Ba tầng có bằng chứng khác nhau.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://helm.sh/docs/

Phụ lục: mã lab để đọc offline

lesson/labs/helm-notes/Chart.yaml

```
apiVersion: v2
name: notes-api
description: Local teaching chart for the Notes API
type: application
version: 0.1.0
appVersion: "lab-1"
```

lesson/labs/helm-notes/templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-notes-api
  labels:
    app.kubernetes.io/name: notes-api
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app.kubernetes.io/name: notes-api
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: notes-api
        app.kubernetes.io/instance: {{ .Release.Name }}
    spec:
      containers:
        - name: notes-api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 2
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 5
            periodSeconds: 10
          resources:
{{ toYaml .Values.resources | indent 12 }}
          volumeMounts:
            - name: lab-data
              mountPath: /data
      volumes:
        - name: lab-data
          emptyDir: {}
```

lesson/labs/helm-notes/templates/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-notes-api
spec:
  selector:
    app.kubernetes.io/name: notes-api
    app.kubernetes.io/instance: {{ .Release.Name }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
```

lesson/labs/helm-notes/values.yaml

```
replicaCount: 1
image:
  repository: notes-api
  tag: lab-1
  pullPolicy: IfNotPresent
service:
  port: 8000
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 250m
    memory: 256Mi
```